

MyOwnCompiler - Day 06

AST Foundation and First Parser

Day 6 Goal

Move from lexical analysis to syntax analysis by building the first parser and the first Abstract Syntax Tree (AST). The compiler now understands program structure instead of only recognizing tokens.

Compiler Evolution

Days 1-5

Source Code

|

v

Lexer

|

v

Tokens

Day 6

Source Code

|

v

Lexer

|

v

Tokens

|

v

Parser

|

v

AST

What is Parsing?

Parsing converts a sequence of tokens into a structured representation of a program. The parser checks whether tokens follow the grammar rules of the language.

Example

Source:

```
print("Hello Parser");
```

Tokens:

```
PRINT  
LEFT_PAREN  
STRING  
RIGHT_PAREN  
SEMICOLON  
EOF  
AST:
```

```
PrintStatementNode  
  |  
  +---- StringLiteralNode  
         |  
         +---- "Hello Parser"
```

Abstract Syntax Tree (AST)

An AST is a simplified tree representation of a program. Compilers use ASTs for semantic analysis, optimization, and code generation.

```
ASTNode
|
+----- StringLiteralNode
```

```
ASTNode
|
+----- PrintStatementNode
```

Why ASTs Matter

Lexer Output:

```
PRINT
LEFT_PAREN
STRING
RIGHT_PAREN
SEMICOLON
```

Parser Output:

```
PrintStatementNode
|
+----- StringLiteralNode
```

Parser Architecture

```
parsePrintStatement()  
  
consume PRINT  
consume (  
consume STRING  
consume )  
consume ;  
  
create StringLiteralNode  
create PrintStatementNode  
return AST
```

Recursive Descent Parsing

Recursive descent parsing is a top-down parsing technique where each grammar rule is represented by a function in code.

Memory Management

```
std::unique_ptr
```

Benefits:

- Automatic cleanup
- No memory leaks
- Clear ownership
- Common in modern compilers

Code Walkthrough

AST Files

ast.h

ast.cpp

Parser Files

parser.h

parser.cpp

Main Flow

Read File

-> Lexer

-> Tokens

-> Parser

-> AST

-> Display Result

Complexity Analysis

Lexing: $O(n)$

Parsing: $O(n)$

Memory:

$O(n)$

Test Case

Input:

```
print("Hello Parser");
```

Output:

Parsed Print Statement

String Value: Hello Parser

Interview Notes

1. What is parsing?
2. What is an AST?
3. Difference between Parse Tree and AST?
4. Why use recursive descent parsing?
5. Why use unique_ptr?
6. Why is parsing needed after lexing?
7. What is top-down parsing?

Revision Sheet (Days 1-6)

Day 1 Project Setup
Day 2 Basic Lexer
Day 3 Operators & Symbols
Day 4 Comments & File Input
Day 5 Strings, Characters, Diagnostics
Day 6 AST Foundation & Parser

Major Components:

Lexer
Token
Parser
ASTNode
StringLiteralNode
PrintStatementNode

Next Milestone

Day 7

ProgramNode
StatementNode
Multiple Statements

Program
|
+-- Print
|
+-- Print
|
+-- Print